

When designing logical decision features for a C-like language, one dilemma is deciding how strict or flexible to be with conditions, should the language allow any nonzero value to be treated as *true* like C, or require an explicit boolean type? For example, supporting `if (x)` is convenient but can hide bugs, while forcing

`if (x != 0)`

improves clarity but reduces familiarity. Another dilemma is how to handle short-circuit evaluation:

`if (a && b)`

should ideally evaluate `b` only if `a` is true, but this requires extra control-flow generation in the compiler. In source form, a user might write:

`if (a && b) { print(1); },`

while the compiler pseudocode may need to expand it into something like:

`t1 = eval(a); if (!t1) goto END; t2 = eval(b); if (!t2) goto END; goto THEN; END:`

Decisions like these affect both language usability and backend complexity, making logical features one of the trickier design areas.